

Documentación del Proyecto A.R.I.A.

A.R.I.A. (Analizador de Reuniones con Inteligencia Artificial) es un programa de escritorio hecho en C++ con la librería Qt 6. El objetivo del proyecto es automatizar el análisis de las transcripciones de las reuniones, usando inteligencia artificial para sacar los participantes, redactar un acta formal, hacer un resumen y anotar las tareas de cada persona.

1. Funcionalidades del programa

- **Análisis con API:** Se conecta con la API REST de Google Gemini 2.5 Flash para mandar y procesar textos largos.
 - **Carga de archivos:** Se pueden cargar las transcripciones arrastrando los archivos a la ventana o buscando archivos con formato .txt o .docx.
 - **Generación de datos:** El programa devuelve de forma automática un resumen breve de la reunión, un acta con los detalles y una lista de tareas para cada participante con su fecha límite.
 - **Exportar a PDF:** Se puede guardar todo el resultado en un archivo PDF usando las clases QPrinter y QTextDocument de Qt.
 - **Base de datos:**
 - **Local:** Guarda el historial de las reuniones en la computadora usando una base de datos SQLite para poder verlo sin internet.
 - **Remota:** Manda una copia de cada análisis a una base de datos MySQL en un servidor VPS usando código PHP, para tenerlo sincronizado.
-

2. Arquitectura del código

El sistema está dividido en distintas capas para no mezclar el diseño visual con la lógica del código:

Interfaz gráfica

- **MainWindow:** Es la clase principal de la ventana y maneja los eventos de los botones y el drag & drop. La regla de diseño que seguimos fue que esta clase no procese datos, solo llama a otras clases y muestra los resultados en la pantalla.

Lógica del programa

- **AlgoritmoIA:** Es una clase abstracta que define los métodos que tiene que tener cualquier clase de IA que agreguemos (como analizar o construirPrompt).
- **AnalizadorGemini:** Es la clase que implementa la IA de Google. Se encarga de armar el archivo JSON y mandarlo por HTTP usando QNetworkAccessManager.
- **ExportadorPDF:** Es una clase que solo se dedica a armar el PDF y guardarlo.

Datos

- **DataManager:** Es una clase tipo Singleton que maneja las conexiones a las bases de datos y mantiene el historial mientras el programa está abierto.

3. Conceptos de POO aplicados

Para el desarrollo del trabajo práctico aplicamos estos conceptos de programación orientada a objetos:

Herencia y clases abstractas

- Hicimos la clase abstracta AlgoritmoIA con métodos virtuales puros como virtual void analizar() = 0. Esto hace que no se pueda instanciar y sirve como molde.
- La clase AnalizadorGemini hereda de esta clase base y escribe el código real para comunicarse con Google. Si después queremos usar ChatGPT, solo hay que hacer una clase nueva que herede de la misma base.

Polimorfismo

- En la clase MainWindow guardamos la IA como un puntero a la clase base haciendo AlgoritmoIA *m_algoritmo.
- La interfaz gráfica no necesita saber qué IA específica se está usando, solo ejecuta el método m_algoritmo->analizar(). Cuando el programa está corriendo, C++ se encarga de llamar al código correcto dentro de AnalizadorGemini.

Static y Singleton

- **Singleton en DataManager:** Para asegurarnos de que no haya dos historiales distintos funcionando al mismo tiempo, esta clase tiene constructores privados y solo se puede acceder llamando al método estático instancia().
 - **Variables Static:** Usamos una variable static en DataManager para llevar la cuenta total de análisis hechos, y en AnalizadorGemini para guardar el nombre del modelo y la URL de la API de forma compartida.
-

4. Guardado de los datos

Para guardar la información implementamos un sistema que escribe en dos bases de datos distintas al terminar el análisis:

1. SQLite (Local):

- Usamos SQLiteDatabase para crear un archivo llamado historial.sqlite directo en la carpeta del proyecto.
- Esto sirve para que el usuario pueda ver lo que analizó antes sin depender de tener internet.

2. MySQL (Servidor remoto):

- Cuando se termina de guardar localmente, la clase DataManager ejecuta el método enviarAlServidor().
 - Esto manda los datos por HTTP POST a un archivo api.php alojado en un VPS, que se encarga de insertar los datos en una tabla de MySQL.
 - *Nota:* Actualmente la base de datos remota guarda los resultados del análisis, pero el objetivo original para implementar este servidor remoto era poder hacer un sistema de login y validación de usuarios (el cual todavía está en desarrollo y no está activo en esta versión).
-

5. Flujo de ejecución

1. **Carga:** El usuario selecciona el archivo de texto y escribe el tema de la reunión en la interfaz.
2. **Llamada polimórfica:** La clase MainWindow agarra esos datos, saca la API key y llama al método analizar() del puntero.
3. **Envío:** AnalizadorGemini arma el pedido en formato JSON y lo manda a Google de forma asíncrona.
4. **Parseo:** Cuando llega la respuesta, el programa lee el JSON y acomoda la información en un struct de C++ llamado ResultadoAnálisis.

5. **Guardado:** Ese struct se le pasa a DataManager, que lo guarda primero en SQLite y al instante lo manda al VPS con MySQL.
6. **Mostrar en pantalla:** La clase AnalizadorGemini emite la señal analisisCompleto. MainWindow atrapa la señal y escribe los textos en las pestañas correspondientes.
7. **Descarga:** Si el usuario quiere, apreta en exportar a PDF y la clase ExportadorPDF se encarga del resto.